

5,8  
6,0

Nome: Julia P. Jaeger 2024 10 358 Data: 07/11/2024

1. Considere uma estrutura de dados baseada em listas encadeadas para gerenciar uma coleção de objetos.

Responda às seguintes perguntas:

a. Explique como uma lista encadeada armazena elementos em memória e como ela difere de um vetor em relação ao modo de alocação de memória.

As listas encadeadas armazenam elementos usando o malloc para criar o endereço de memória do tamanho necessário. Por essa razão as listas encadeadas se diferem de vetores, não é preciso definir o número de elementos que vão existir no início. E possui um fácil acesso, já que estão todos conectados (prox e ant, se duplamente encadeado).

b. Discuta as vantagens e desvantagens do uso de listas encadeadas em relação a vetores para operações de:

i. Inserção de um elemento no início da lista

Vantagem: na lista encadeada vai criar um espaço de memória apenas quando for inserir, (não terá excesso). E apenas precisa alocar o prox no nó cabeça, não precisa percorrer.

Desvantagem: no vetor, teria que passar todos os elementos uma casa para frente, para liberar o espaço para novo elemento (+ trabalho).

ii. Inserção de um elemento no final da lista

Vantagem: Basta apenas alocar o novo elemento lá no prox do último da lista.

Desvantagem: É necessário percorrer toda a lista, enquanto no vetor poderia ser tipo:  $\text{Vel}[5] = n$ ;

iii. Acesso direto a um elemento específico (exemplo: acessar o quinto elemento da lista)

Vantagem: É mais fácil a realocação em uma lista encadeada, no caso de uma remoção não há necessidade de ant-prox no n-prox e o prox-ant no n-ant e daria um free() (libera a memória).

Desvantagem: Enquanto isso em um vetor faria  $\text{Vel}[4]$ , na lista seria necessário percorrer toda ela.

c. Para uma aplicação em que os dados são constantemente removidos e adicionados de forma aleatória (não ordenada), justifique por que uma lista encadeada seria ou não mais apropriada que um vetor.

A lista encadeada é mais apropriada pois não terá excesso do uso de memória. Vai criar um endereço (malloc) quando necessário inserir e dar um free(), para liberar o endereço quando não mais necessário.

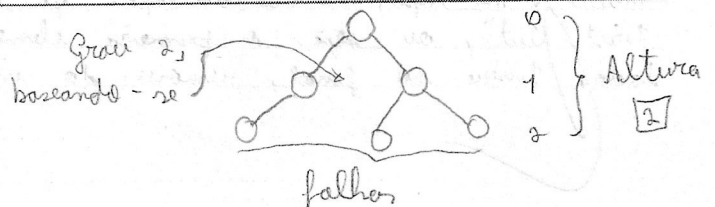
2. Com relação ao conteúdo de árvores binárias estudado durante o semestre, responda:

a. Descreva o que é nó folha, grau e altura de uma árvore:

Nó folha: nó que não possui nenhum filho;

Grau: Número de filhos que um nó pode ter.

Altura: Tamanho de "linha" que uma árvore tem, desde o nó raiz até a folha mais longe.



0,2 b. Descreva as formas de percurso utilizadas em árvores:

Pre-ordenado: percorre a raiz, depois o nó a esquerda, depois o nó a direita.

Simétrico: percorre o nó a esquerda, depois a raiz, depois o nó a direita.

Pos-ordenado: percorre o nó a esquerda, depois o nó a direita e por último a raiz.

0,2 c. Explique como a operação de remoção em uma árvore de busca binária é realizada e quais são os casos a serem considerados.

A ~~operação~~ operação de remoção é feita verificando primeiramente qual o caso a ser utilizado, se se quer remover uma folha (apenas remove o nó e o pai recebe NULL no lugar onde o endereço a filha estava), se o nó que quer remover possui apenas um filho (colo o filho do nó no pai e vice-versa e retorna) e se o nó tem + de 1 filho (Então o novo pai ou será o nó mais da esquerda, ou será necessário uma verificação mais complexa).

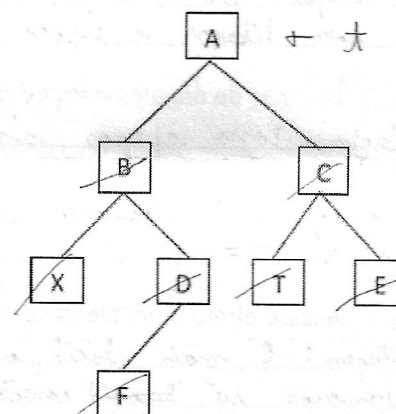
3. Analisando o código abaixo e a representação hierárquica da árvore binária, mostre qual a saída correta após a execução da função. Assuma que a chamada da função *insert* retorna um ponteiro para a raiz da árvore.

0,4

```

9 #include <stdio.h>
10
11 struct tree {
12     char info;
13     struct tree *left;
14     struct tree *right;
15 };
16 typedef struct tree Tree;
17
18 void print(Tree *t) {
19     if (t != NULL) {
20         if (t->left != NULL)
21             print(t->left);
22         if (t->right != NULL)
23             print(t->right);
24         printf("%c", t->info);
25     }
26 }
27
28 int main() {
29     Tree *t = NULL;
30     t = insert(t);
31     print(t);
32     return 0;
33 }

```



Saída: X F D B T E C A

0,5 4. Estruturas de dados do tipo pilha e fila diferem apenas na ordem de remoção de seus elementos. A inserção é realizada seguindo o mesmo critério. Essa afirmação está correta? Justifique.

Não completamente. As pilhas seguem o caso "Last in, First Out", ou seja, o último elemento a ser inserido, será o primeiro manipulado. Insere no topo, remove do topo. Já as filas seguem o caso "First In, First Out", ou seja, o primeiro elemento inserido, será o primeiro manipulado. Insere no final, remove do início.

0,5 5. Analise o código a seguir e diga o que será apresentado na linha 12. Justifique sua resposta:

```
1  #include <stdio.h> 5 4 3 2 1 0
2  int recursivo(int n) {
3      if (n <= 0) {
4          return 0;
5      } else {
6          n--; 4 3
7          return n + recursivo(n);
8      }
9      4 + 3 + 2 + 1
10
11 int main(){
12     printf("%d", recursivo(5));
13     return 0;
14 }
```

| $n$ | $n--$ | return                |
|-----|-------|-----------------------|
| 5   | 4     | $4 + 6 = 10$          |
| 4   | 3     | $3 + 3 = 6$           |
| 3   | 2     | $2 + 1 = 3$           |
| 2   | 1     | $1 + 0 = 1$           |
| 1   | 0     | $0 + 0 = 0$           |
| 0   |       | 6 não entra, return 0 |

0,3 Será apresentado na linha 12 o valor 10. A função recursiva altera o número de parâmetro e  $n < 0$ , caso não, apenas retorna 0. Entretanto, caso  $n > 0$ , é subtraído em 1 e somado seu valor com o retorno da próxima chamada, conforme a tabela a cima:

0,3 6. Indique se cada uma das afirmações abaixo é verdadeira (V) ou falsa (F). Justifique as falsas.

X (F) Em uma árvore binária de busca, um nó pode ter apenas um filho esquerdo ou um filho direito, mas nunca ambos. Por que?

Resposta correta, faltou justificar (V) A recursão pode ser utilizada para simplificar a implementação de algoritmos que operam em estruturas de dados como árvores e grafos.

X (F) O número de chamadas recursivas em uma função não afeta o uso de memória. Por que?

(V) Uma função recursiva que não possui um caso base pode resultar em uma pilha de chamadas que nunca é liberada.

(V) Uma árvore binária cheia sempre é também uma árvore estritamente binária.

1,5 7. Assumindo duas listas encadeadas não vazias que armazenam valores inteiros, desenvolva um algoritmo para verificar quantas vezes cada elemento da primeira lista aparece na segunda lista. Mostrar também os endereços de memória desses elementos na segunda lista. Não há necessidade de criar o registro nem as funções de inserção e main. O protótipo da função deve ser o seguinte:

void contaOcorrencias (Lista \*L, Lista \*L2);

void contaOcorrencias (Lista \* L, Lista \* L2) {

int count = 0;

Lista \* aux = L2;

if (L != NULL) { ~~if (aux != NULL)~~

while (aux != NULL) {

if (aux->val == L->val) {

count++;

printf("Encontrado no endereço %x", aux);

aux = aux->prox;

printf("O número %d foi encontrado %d vezes", L->val, count);

if (L->prox != NULL) {

contaOcorrencias (L->prox, L2);

}